

> Session 1

Setup, first programs, and reading errors

Dr. Ramon Winterhalder

Human-Centered Artificial Intelligence · A.Y. 2026/27



UNIVERSITÀ
DEGLI STUDI
DI MILANO



> How this lab works



...a typical block, three or four per session, while some run longer. About 80% of the time you are at the keyboard.

Every exercise has a level

- core** everyone — finish these first
- stretch** when you are done with core
- challenge** if you have programmed before

Predict first, then run.

A wrong prediction you understand is worth more than a lucky right one.

› Nine sessions, two lecturers

1

Setup · running Python ·
first programs · errors

2

Variables · values · types
· operators · input

3

Conditions · logic · try /
except

4 – 9

Loops, lists, strings,
dictionaries,
functions, files,
libraries

Sessions 1–3: Ramon Winterhalder **Sessions 4–9:** *TBC*

Running thread in sessions 1–3: you build one small program, a rule-based study assistant, a little more each week.

› Also taking “Programming”?

Programming (course)

- Understand and predict
- Trace code on paper, explain why
- Written exam, graded out of 30
- 6 CFU · 48 h

Programming Lab (this)

- Build, run, test, debug
- At the keyboard, most of the time
- Pass / fail
- 3 CFU · 36 h

Same language, some of the same topics — but a different job. Expect overlap, but the exercises are different.

› Passing the lab

Standard route

- A final practical test at the end of the lab
- Format to be announced
- Nothing to hand in during the term
- The core exercises are the preparation

Project route

- For students who can already program
- Evidence: a passed programming exam, or a GitHub repo with your own code
- Individual project, ~10 min presentation + questions on your code, at the end
- **Decide by Session 3 (12.10)**

Details of both routes will be confirmed in the coming weeks.

› Where to find the material

myAriel

The official course page: announcements and all material, sessions 1–9.



myariel.unimi.it/user/index.php?id=14344

GitHub

Slides, exercise notebooks and solutions for sessions 1–3. No login needed.



github.com/ramonpeter/programming-lab-hcai-2026

No access to Ariel yet? Use GitHub. Everything for sessions 1–3 is there.

> Today

1	Setup	Python, VS Code, notebooks — on your machine	60 '
2	Three ways to run Python	prompt, script, notebook · the terminal	45 '
	<i>Break</i>		15 '
3	Python as a calculator	print, operators, text vs numbers	30 '
4	When things go wrong	reading error messages	40 '
5	Mini-project	study assistant v0.1	35 '

01

Setup

Three pieces of software, and a folder to keep your work in

> What you are installing

Python

the interpreter

reads your code and runs it

VS Code

the editor

where you write code; has a terminal built in

Jupyter

notebooks

code + output + text in one .ipynb file — runs inside VS Code

Google Colab — notebooks in the browser, nothing to install. **Your fallback today if installation fails.** Not a replacement: you cannot run a .py script from a terminal there, and we will.

> Your route — full steps in SETUP.md

Windows

- Python install manager
- from python.org or MS Store
- answer yes to its questions

```
PS> py install 3.14  
PS> python --version
```

macOS

- python.org installer
- then Install Certificates
- use python3, not python

```
$ python3 --version
```

Linux

- already installed, usually
- else: apt install python3
- use python3, not python

```
$ python3 --version
```

Then, everyone: VS Code + the Python and Jupyter extensions (by Microsoft).

Expert mode: conda (Miniconda / Anaconda) instead of python.org — fine, but not both.

EXERCISE A

🕒 45 min · `notebook lab01-exercises.ipynb`

- A1** **core** Install Python + VS Code (+ Python & Jupyter extensions). Tick all five boxes in the notebook — the last one: run `print("hi")` in Colab.
- A2** **core** Create a folder `programming-lab/lab01`. Put the notebook and `broken.py` there. In VS Code: File → Open Folder... → `programming-lab`.

Stuck? Put your hand up — we solve it now.

Still not working after ~20 minutes: Colab for the rest of today, finish the setup at home before session 2.

first cell of the notebook – you are done when it prints
Python 3.14.x on Windows

02

Three ways to run Python

The prompt, a script, a notebook — and the terminal that connects them

› Same Python, three front doors

Prompt (REPL)

```
$ python3  
>>> 2 ** 10  
1024  
>>> exit()
```

try one line, get an answer

Script .py

```
# hello.py  
print("Hi")  
  
$ python3 hello.py  
Hi
```

a program: runs top to bottom

Notebook .ipynb

```
# a cell  
print("Hi")  
2 ** 10  
  
Hi  
1024
```

cells, output kept in the file

A script shows only what you print. A notebook also shows the value of the last line — a convenience, not Python.

> Who are you talking to?

```
$ python3 hello.py
Hi
$ python3
>>> 2 + 2
4
>>> exit()
$
```

`$` `%` `C:\...>` the terminal — runs commands

`>>>` Python — runs Python

Five commands are enough

`pwd` where am I?

`ls` (Windows: `dir`) what is in this folder?

`cd lab01` go into a folder

`cd ..` go up one level

`python3 file.py` run a script (Windows: `python`)

EXERCISE B🕒 25 min · `notebook lab01-exercises.ipynb`

- B1** **core** Open the Python prompt. Compute `2 ** 10` and `17 / 4`. Then type `python --version` at `>>>`. What happens, and why?
- B2** **core** Create `hello.py`: one `print` line and a bare `2 ** 10`. Run it with  and from the terminal. Where is the 1024? Now run the same two lines in a notebook cell.
- B3** **core** From one folder up (`cd ..`), try `python hello.py`. Read the error. Fix it two ways: with the path, and with `cd`.
- B4** **stretch** A cell with `3 + 4`, a `print`, and `10 * 10`: what is displayed?
- B5** **challenge** What do `python -c "print(6*7)"` and `python -i hello.py` do?

Before you ask for help, look at the prompt: `>>>` means Python; `$`, `%` or `C:\>` means the terminal.

03

Python as a calculator

print, operators, and the difference between text and numbers

> Operators, and printing an answer

+ -

add, subtract

multiply

/

divide → always a decimal

//

whole times it fits

%

remainder

power

*Brackets first, then **, then * // %, then + -*

```
print("Usable seats:", 14 * 22 - 9)
Usable seats: 299
```

```
print("14 * 22")
14 * 22
```

```
print("7" + "3", "ha" * 3)
73 hahaha
```

Quotes make text. Text is shown, not computed — and + and * mean something else for text.

EXERCISE C

🕒 20 min · `notebook lab01-exercises.ipynb`

- C1** core Ten lines: write your prediction as a comment, then run. Mark every line where you were wrong.
- C2** core Four real questions, one print each, Python does all the arithmetic: seats in a hall, hours of lab time, download minutes, groups of 6.
- C3** stretch Print a banner with `"=" * 40`. What does `sep=` do in `print`?
- C4** challenge `2 ** 100` vs `2.0 ** 100` — why do they look different? How many years is `2 ** 64` seconds?

Predict first, then run. Wrong predictions are the useful ones: write down why you were wrong.

04

When things go wrong

Error messages are instructions. Learn to read them from the bottom up.

➤ Reading an error message

```
Traceback (most recent call last):  
  File "assistant.py", line 7, in <module>  
    pritrn("Total hours:", 50)  
    ^^^^^  
NameError: name 'pritrn' is not defined. Did you mean: 'print'?
```

1 Last line

the kind of error, and what
went wrong

2 Line number

where Python noticed it

3 ^^^^^

the exact spot in that line

Python often tells you the fix. Read it before you ask anyone — including an AI.

> Two kinds of error

Python can't read it

SyntaxError

```
print("Total:" 50)
SyntaxError: invalid syntax.
Perhaps you forgot a comma?
```

The file never starts. Not even the correct lines above run.

Python can't do it

NameError, TypeError, ZeroDivisionError ...

```
print("Start")
print(50 / 0)
Start
ZeroDivisionError: division by zero
```

Runs line by line, then stops at the problem. Earlier output stays.

And a third, worse kind: no error at all, and a wrong answer. Always ask: is this plausible?

EXERCISE D

🕒 25 min · `notebook lab01-exercises.ipynb`

- D1** **core** Five broken lines. Run each, name the error from the last line, fix it.
- D2** **core** `broken.py` has four mistakes. Fix one, run, repeat. Before each run: which lines will print? Why did nothing print on the first two runs?
- D3** **stretch** Write four one-liners that each produce a different error: `SyntaxError`, `NameError`, `TypeError`, `ZeroDivisionError`.
- D4** **challenge** `print(2 ^ 10)` prints 8. Look up `^` in the Python docs and explain.

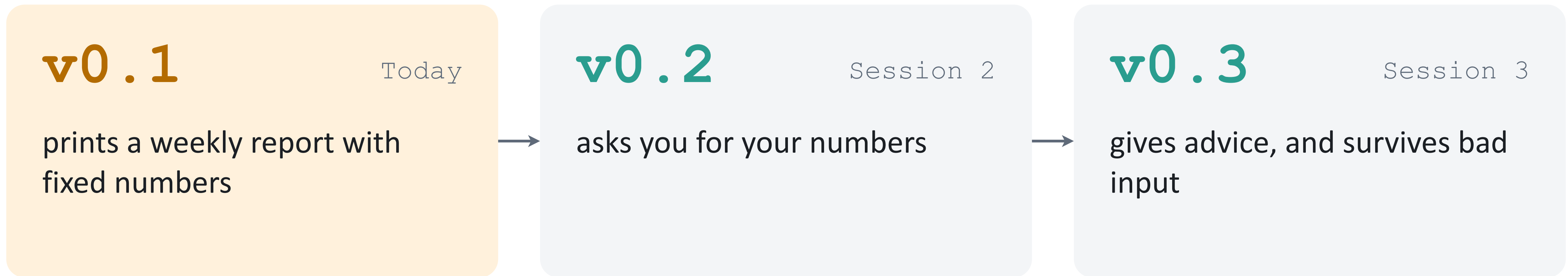
Last line first, then the line number, then the `^^^^`. Fix one thing, run again.

05

Mini-project

A study assistant that only follows rules — version 0.1

> assistant.py — three versions



Rule-based, not learned: it does exactly what its rules say. The first chatbot, ELIZA (1966), worked like this.

- It is a .py script, run from the terminal — not a notebook
- One file, in your lab01 folder which you extend each week

EXERCISE E

🕒 25 min · notebook lab01-exercises.ipynb

E1**core**

Create `assistant.py` with three comment lines at the top (name, date, what it does). Make it print the report on the right. 5 lectures $\times$ 4 h; every number computed.

E2**stretch**

`help(round)` — then print hours per day as 7.1.

E3**challenge**

How often did you type `5 * 4`? Next week has 6 lectures: what could go wrong? Answer as a comment.

```
=====
      STUDY ASSISTANT  v0.1
=====
Hello! I am a very simple
assistant.
I do not learn. I only follow rules
someone wrote.

This week (fixed numbers, for now):
Lecture hours: 20
Self-study hours (1.5 per
    lecture hour): 30.0
Total hours: 50.0
Hours per day (7 days): 7.14285...
=====
```

➤ Today you...

- installed Python and ran it three ways
- moved around with the terminal
- used Python as a calculator
- read an error message from the bottom up
- wrote your first script

Next · Session 2

Variables, types, operators, and asking the user for input.

Before then: make sure your setup works.
Nothing to hand in.

No laptop today, or setup not finished? That is your homework — follow SETUP.md.